

Proposta de uso de LLM para apoio ao diagnóstico: triagem automática de logs de execução

Lucas Camaz Ferreira

1 Contexto

A rotina de coleta (`g1_scraper`) já produz, para cada execução, um log em dois níveis (console em INFO, arquivo em DEBUG) e um resumo final com métricas agregadas (páginas coletadas, páginas com falha, registros salvos, duplicados descartados, anúncios filtrados, campos ausentes, código de saída). Quando a coleta apresenta uma anomalia — poucos registros, base vazia, código de saída 1 — o primeiro passo do diagnóstico hoje é ler esse log manualmente para decidir se a causa é *interna* (falha de rede, timeout, bug de parsing) ou *externa* (mudança no contrato da API). Esse é exatamente o gargalo que apareceu na prática durante o desenvolvimento: um HTTP 400 acidental (corpo da consulta enviado como objeto em vez de lista) só foi identificado porque alguém leu manualmente dois registros de `ERROR` com status e parâmetro `from`, cruzados com o resumo final (0 registros na base | 0 páginas ok, 2 com falha).

A proposta a seguir usa uma LLM para automatizar essa primeira triagem, sem entregar a ela nenhuma decisão automática sobre o código.

2 Em que etapa e com qual função

O modelo é acionado **depois** que uma execução termina com algum sinal de anomalia (código de saída diferente de 0, taxa de páginas com falha acima de um limiar, ou registros salvos abaixo do esperado) e **antes** de qualquer investigação manual.

A função do modelo é estritamente de **triagem**: ler as linhas de log relevantes e o resumo da execução e devolver uma classificação com evidência textual — causa provável, módulo suspeito e o trecho do log que justifica o veredito. O modelo não corrige código, não decide sozinho e não é acionado durante a coleta de rotina; ele participa apenas do ciclo de diagnóstico, como um assistente de primeira leitura.

3 Quais dados seriam fornecidos ao modelo

- As linhas de nível `WARNING/ERROR` do log `DEBUG` da execução com anomalia (log já filtrado por nível, não o arquivo inteiro).
- O resumo final produzido pelo `__main__.py`: páginas coletadas, páginas com falha, registros, duplicados, anúncios filtrados, campos ausentes, código de saída.
- O mesmo resumo de uma ou duas execuções anteriores bem-sucedidas (extraído do manifesto em `data/output/`), como linha de base para comparação.
- Quando existir, o *traceback* completo de uma exceção não tratada.

Note-se o que **não** é enviado: nenhum trecho de HTML, nenhum hit do JSON da API, nenhum seletor. A entrada é só texto de log e números de manifesto — isso já restringe bastante o espaço

em que o modelo poderia “inventar” algo, porque não há dado de domínio para especular sobre estrutura de página ou de resposta.

4 Como as respostas seriam validadas

A saída é forçada a um schema fixo, validado com Pydantic antes de qualquer uso:

Listing 1: Schema de validação da resposta do modelo

```
from pydantic import BaseModel
from typing import Literal, Optional

class TriagemLog(BaseModel):
    causa_provavel: Literal["interno", "externo", "indeterminado"]
    modulo_suspeito: Optional[
        Literal["client.py", "parser.py", "scraper.py", "storage.py"]
    ]
    evidencia: str # trecho literal do log fornecido
    confianca: Literal["baixa", "media", "alta"]
```

A validação central não é semântica, é sintática e barata de checar automaticamente: o campo **evidencia** precisa ser uma substring que existe literalmente no texto de log enviado ao modelo. Se a checagem falhar, a resposta inteira é descartada e o resultado é tratado como **indeterminado** — nunca chega a uma pessoa como triagem válida.

A saída nunca aciona nada sozinha: ela vira um comentário anexado à execução (ou a uma issue), para leitura humana. Não existe caminho automático entre essa triagem e uma mudança no código-fonte.

5 Mecanismos contra alucinação

- **Vocabulário fechado:** `causa_provavel` e `modulo_suspeito` são enumerações fixas (schema Pydantic), não texto livre — o modelo não pode citar um módulo que não existe no projeto.
- **Evidência obrigatória e verificável:** todo veredito exige uma citação literal do log de entrada; resposta sem essa citação, ou cuja citação não é encontrada por busca de substring no texto original, é rejeitada automaticamente.
- **Sem acesso a nada além do que foi anexado ao prompt:** o modelo não consulta a API ao vivo, não executa código, não lê o repositório — apenas o log e o resumo colados na chamada. Isso fecha a porta para o modelo “completar” informação ausente com conhecimento geral.
- **Temperatura baixa** (próxima de 0): a tarefa é de classificação/extração, não de geração criativa, o que reduz variância entre chamadas com a mesma entrada.
- **indeterminado como saída legítima:** o schema permite explicitamente o modelo declarar incerteza em vez de ser forçado a escolher entre interno/externo quando a evidência é fraca, reduzindo o incentivo a forçar uma causa plausível sem base real.

6 Pseudocódigo do ciclo

Algorithm 1 Triagem automática de log de execução com anomalia

Require: log de execução L , resumo da execução R , resumos de referência R_{ref}

Ensure: classificação validada ou **indeterminado**

```
1:  $anomalia \leftarrow \text{DETECTARANOMALIA}(R, R_{\text{ref}})$ 
2: if not  $anomalia$  then
3:   return NULO ▷ execução normal, modelo não é acionado
4: end if
5:  $L_{\text{filtrado}} \leftarrow \text{FILTRARPORNIVEL}(L, \{\text{WARNING}, \text{ERROR}\})$ 
6:  $prompt \leftarrow \text{MONTARPROMPT}(L_{\text{filtrado}}, R, R_{\text{ref}})$ 
7:  $resposta \leftarrow \text{CHAMARLLM}(prompt, \text{temperatura} = 0)$ 
8:  $schema\_ok \leftarrow \text{VALIDARSCHEMAPYDANTIC}(resposta)$ 
9: if not  $schema\_ok$  then
10:  return indeterminado ▷ resposta mal formada é descartada
11: end if
12:  $evidencia\_ok \leftarrow \text{SUBSTRINGEXISTEEM}(resposta.evidencia, L_{\text{filtrado}})$ 
13: if not  $evidencia\_ok$  then
14:  return indeterminado ▷ evidência não verificável no log original
15: end if
16:  $\text{ANEXARCOMENTARIOPARAREVISAOHUMANA}(resposta)$ 
17: return  $resposta$  ▷ nunca dispara ação automática no código
```
